UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA INGENIERÍA EN SOFTWARE

TEMA:

FICHAS BIBLIOGRÁFICAS: FUNDAMENTOS DE SISTEMAS DISTRIBUIDOS Y ARQUITECTURAS DISTRIBUIDAS

PRESENTADO POR:

HERRERA BARCO HUMBERTO ALDAIR

FECHA:

VIERNES, 6 DE NOVIEMBRE DEL 2026

(*Semana 02*)

Ficha bibliográfica 01

Título: Systematic Analysis of DDS Implementations

DOI: 10.1145/3590140.3629118

Resumen:
El artículo toma como base la dificultad de elegir entre implementaciones de DDS, ya que existen parámetros que no están cubiertos por el estándar y las comparaciones no siempre son justas [1]. Para resolverlo, presenta DDS-Perf, una herramienta que permite comparar el rendimiento mediante pruebas y condiciones similares. Se evaluaron cuatro implementaciones en un clúster de Raspberry Pi, midiendo aspectos como el ancho de banda, la latencia y la tasa de mensajes. Los resultados muestran que RTI Connext destaca en ancho de banda y tasa de muestras, mientras que FastDDS obtiene resultados en latencia. Sin embargo, ningún proveedor sobresale en todos los aspectos [1].

Dato concreto:
Un dato que se me hizo interesante fue en la prueba 1-to-1, entre cuatro implementaciones, RTI Connext y a FastDDS, en la cual RTI Connext alcanzó un ancho de banda mediano de 157 Mbps, mientras que FastDDS obtuvo la menor latencia de extremo a extremo, con 2,84 ms. Me interesó porque se puede ver que el rendimiento de una tecnología de comunicación distribuida debe evaluarse considerando diferentes métricas, no solamente la velocidad de transmisión.

Ubicación: páginas 5 y 6; sección 4.1, Benchmark: 1-to-1.

Nota personal:
Este artículo me ayudó a entender que en un sistema distribuido la comunicación no depende solo de los mensajes, sino también del middleware y su rendimiento. Aprendí que las implementaciones de DDS pueden variar mucho en ancho de banda y latencia, así que hay que elegir la que mejor se ajuste a lo que necesita el sistema. Me va a servir en la TA-IND-01 para analizar cómo se comunican los componentes del sistema que elija y qué tan importantes son el rendimiento y la confiabilidad.

Ficha bibliográfica 02

Título: Benchmarking scalability of stream processing frameworks deployed as microservices in the cloud

DOI: 10.1016/j.jss.2023.111879

Resumen:
En este articulo se menciona la falta de estudios que comparen la escalabilidad de distintos frameworks de procesamiento de flujos. Para ello, evalúa herramientas como Flink, Kafka Streams, Hazelcast Jet y Beam mediante pruebas en Kubernetes, utilizando más de 740 horas de experimentos. Los resultados muestran que todos pueden escalar si cuentan con recursos suficientes, pero su rendimiento depende del tipo de trabajo [2]. Ningún framework es superior en todos los casos y Beam presenta mayores requisitos. También se destaca que el escalado vertical puede complementar al horizontal y que elegir un framework adecuado influye directamente en los costes de operación [2].

Dato concreto:
En la prueba base del artículo con el benchmark UC3, Hazelcast Jet necesitó una sola instancia para procesar todas las cargas evaluadas (hasta 100 000 sensores simulados), mientras que Flink requirió hasta 10 TaskManagers y Kafka Streams hasta 18 instancias. Me interesó porque en el benchmark UC4 el resultado se invierte: Hazelcast Jet tuvo la mayor demanda de recursos, y ni con 30 instancias pudo manejar la carga de más de 7 grupos anidados. Esto muestra que no hay un framework superior en todo, y que el resultado depende del caso de uso.

Ubicación: páginas 6 y 7; sección 6.1, Baseline comparison of frameworks.

Nota personal:
Este artículo me ayudó a entender cómo se mide si un sistema distribuido aguanta cuando sube la carga. Aprendí que escalar no es solo meter más instancias, también importa cuántos recursos necesita cada framework, y que no hay uno que sea el mejor en todo, depende del uso. Esto me servirá en la TA-IND-01 para analizar cómo escala el sistema que elegí y qué necesita para seguir funcionando cuando crezca la demanda.

Ficha bibliográfica 03

Título: Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution

DOI: `10.1007/s00607-026-01701-5`

Resumen:
El estudio aborda la falta de una comprensión unificada sobre cómo mantener la consistencia en arquitecturas de microservicios orientadas a eventos. Para investigarlo, los autores realizaron una revisión de literatura, en la que analizaron 38 estudios seleccionados de un total de 119 documentos [3]. Los resultados permitieron identificar tres tipos de consistencia: datos, procesos y evolución del sistema. También se encontraron diferentes estrategias para abordar estos problemas, como el patrón Saga, las transacciones, la orquestación y la coreografía. Finalmente, se concluye que las soluciones existentes suelen tratar estos problemas por separado y que aún existe una brecha entre el diseño y el funcionamiento real de los microservicios [3].

Dato concreto:
Un dato que se me hizo interesante es que, al clasificar los estudios revisados en tres tipos de consistencia, la consistencia de datos reunió 12 documentos, frente a 7 de consistencia de procesos y 6 de consistencia de evolución del sistema. Los autores indican que la consistencia de datos fue el desafío más enfatizado en la literatura. Me interesó porque muestra que, en microservicios basados en eventos, el problema más estudiado es mantener coherente el estado entre servicios distribuidos, y no solo lograr que se comuniquen entre sí.

Ubicación: página 16; sección 6.2.1, Data consistency, y Tabla 4 (la clasificación se introduce en la sección 6.1, página 15)

Nota personal:
Este artículo me ayudó a entender que la "consistencia" en microservicios no es un solo problema, sino tres: la de los datos, la de los procesos y la de cómo evoluciona el sistema. Aprendí que de 119 documentos encontrados los autores se quedaron con 38 estudios recientes, y que

la mayoría se enfoca en que los datos no se corrompan cuando los microservicios trabajan de forma distribuida. Esto me servirá en la TA-IND-01 para ver qué problemas de consistencia puede tener el sistema que elegí y cómo se podrían resolver.

Ficha bibliográfica 04

Título: Publish/Subscribe for Edge Intelligence: Systematic Review and Future Prospects

DOI: 10.1145/3849800

Resumen:
En este artículo se mencionan los problemas de comunicación que generalmente surgen con el crecimiento del Internet de las Cosas, donde millones de dispositivos generan grandes cantidades de información que deben procesarse de forma rápida [4]. Debido a sus limitaciones de memoria, capacidad y batería, muchos dependen de servidores externos para poder realizar estas tareas. Los autores señalan que los métodos tradicionales de comunicación no responden muy bien a las necesidades actuales. Por ello, estudian el modelo Publish/Subscribe, que permite que unos dispositivos publiquen información y otros reciban únicamente los mensajes que les interesan, sin necesidad de conocerse entre ellos [4].

Dato concreto:
En la revisión se identificaron 237 artículos según los criterios de inclusión y exclusión, y de ellos se seleccionaron 191 para la evaluación de calidad.

Ubicación: página 16:9; sección 3.1 (Define inclusion and exclusion criteria) y sección 3.2 (Identify the primary studies); resumen del proceso de selección en la Figura 1 (página 16:8).

Nota personal:
Me ayudó a ver cómo mejorar la comunicación entre dispositivos y servicios en un sistema repartido. Para mi proyecto TA-IND-01, el modelo Publish/Subscribe me parece buena opción para que los componentes se comuniquen sin hablar directamente entre todos. También aprendí que hay que elegir bien el protocolo según la rapidez, la capacidad de crecer, la seguridad y los recursos, y me llamó la atención la idea de usar inteligencia artificial para que el sistema se adapte solo a la red.

Ficha bibliográfica 05

Título: A Survey on Networked Data Streaming With Apache Kafka

DOI: 10.1109/ACCESS.2023.3303810

Resumen:
El artículo habla sobre el uso de Apache Kafka para procesar grandes cantidades de datos en tiempo real y cómo su rendimiento depende de la configuración. Para estudiarlo, los autores realizaron una revisión de más de 90 trabajos, de los cuales seleccionaron hasta 70 para analizar. Los resultados destacan que Kafka permite distribuir los mensajes y dividir los datos para

procesarlos al mismo tiempo [5]. Además, guardar copias ayuda a mantener el funcionamiento ante fallos y permite integrarse con herramientas como Spark, Storm y Flink. Se aplica en diferentes sectores, aunque todavía existen retos relacionados con la seguridad, la rapidez y la escalabilidad [5].

Dato concreto:
En la revisión se encontraron más de 90 artículos, y tras aplicar el criterio de exclusión de falta de una contribución sólida sobre Apache Kafka, el número final de trabajos considerados fue de hasta 70.

Ubicación: página 3; sección II (Methodology and Literature Overview), paso 3 (Study selection).

Nota personal:
Este artículo siento que me va a servir mucho para mi proyecto TA-IND-01 porque explica cómo usar Kafka para que las partes de un sistema se hablen entre sí y se enteren de todo al instante. Me quedó claro cómo viajan los datos, cómo se organizan y cómo las copias evitan que todo se caiga si algo falla. Además, muestra ejemplos reales en fábricas y dispositivos conectados, así que me puede ayudar a decidir cómo armar la comunicación en mi proyecto.

Declaración de uso de inteligencia artificial

Herramienta utilizada: Claude (Anthropic).

Para qué se usó: La herramienta se utilizó únicamente para corregir la redacción y los errores ortográficos de los textos, traducir fragmentos de los artículos consultados y escribir de forma correcta el código LateX del documento.

Cómo se verificó lo obtenido: Los títulos, autores, DOI y cifras citados se contrastaron con los artículos originales en PDF. Las traducciones y correcciones de redacción se revisaron para confirmar que no cambiaran el sentido del texto original. Finalmente, el documento se compiló y se revisó completo, incluida la lista de referencias, antes de la entrega.

Referencias

[1] V. Bode, D. Buettner, T. Preclik, C. Trinitis, and M. Schulz, "Systematic analysis of DDS implementations," in *Proceedings of the 24th International Middleware Conference (Middleware '23)*. Bologna, Italy: ACM, Dec. 2023, pp. 234–246.

[2] S. Henning and W. Hasselbring, "Benchmarking scalability of stream processing frameworks deployed as microservices in the cloud," *Journal of Systems and Software*, vol. 208, p. 111879, Feb. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121223002741

[3] J. L. Bação and S. Guerreiro, "Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution," *Computing*, vol. 108, no. 7, p. 110, Jun. 2026. [Online]. Available: https://doi.org/10.1007/s00607-026-01701-5

[4] A. Saleh, S. Tarkoma, S. Pirttikangas, and L. Lovén, "Publish/subscribe for edge intelligence: Systematic review and future prospects," *ACM Computing Surveys*, vol. 59, no. 1, Oct. 2026. [Online]. Available: https://doi.org/10.1145/3849800

[5] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.